SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1021	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	P. Siddardha	**Reg. No.:**	192424271
Date:	15-10-2026	**Duration:**	60 minutes

Code of Conduct

I p.siddardha (192424271) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. Siddardha

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. **Understand the Problem Statement**
   - Explain the problem in your own words.
   - Identify the objectives of the proposed system.
   - State the scope of the problem.
2. **Identify Key Stakeholders and Challenges**
   - Identify all the stakeholders involved in the system.
   - Explain the roles and responsibilities of each stakeholder.
   - Discuss the major challenges and issues faced by the stakeholders.
3. **Analyze the Current Approach**
   - Describe the existing system or current process.
   - Identify its strengths and limitations.
   - Analyze the problems or gaps that need to be addressed.
4. **Propose a Suitable Solution Using Software Engineering Principles**
   - Design a software-based solution for the given problem.
   - Describe the major functional modules of the proposed system.
   - Identify the functional and non-functional requirements.
   - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
5. **Justify the Chosen Methodology/Framework**

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. **Discuss Expected Outcomes and Limitations**
   - Describe the expected benefits of the proposed solution.
   - Explain how the solution addresses the identified challenges.
   - Discuss potential risks, implementation challenges, and limitations.
   - Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework	Selects the most suitable	Suitable methodology	Methodology selected but	Outcomes, limitations, or

Justification (10%)	SDLC/model/frame work and provides strong technical justification.	selected with adequate justification.	justification is weak.	future enhancements are missing; report lacks organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

Industry Context

Modern defense operations depend heavily on Low-Earth Orbit (LEO) satellite constellations to route secure communications, scan terrain, and track logistics. However, because these satellites cross international borders and communicate over wireless links, they are prime targets for state-sponsored cyberattacks. Satellite networks need an automated cryptographic key management fabric that rotates security keys across thousands of fast-moving spacecraft safely.

Problem Statement

Design and develop a cryptographically secure, high-integrity LEO Satellite Key Management Fabric that updates and secures data links across global satellite constellations. The system must handle high-volume key generation tasks, distribute encrypted updates to satellites during tight contact windows, verify node identities, and isolate compromised satellites automatically.

Objectives

- Secure the data lifecycle of cryptographic keys from ground control bases to orbital nodes.
- Implement fast, lightweight key update workflows that fit within short satellite pass windows.
- Rotate satellite access keys automatically based on dynamic orbital location risks.
- Isolate compromised satellite nodes instantly to prevent lateral cyber-breaches across the fleet.
- Maintain tamper-proof, unalterable system logs of key access steps for security audits.

Expected Outcomes

- Absolute security and anti-hacking protection for high-value military satellite networks.
- Elimination of manual key management overheads via fully automated orbital updates.
- Near-zero data leak risks across distributed multi-satellite communication pipelines.
- High system operational reliability across highly fragmented global tracking channels.

1. Understand the Problem Statement

1.1 Problem in My Own Words

Low-Earth Orbit (LEO) satellites are used for communication, observation and other important services. These satellites move rapidly around the Earth and communicate with ground stations and other authorized systems through wireless links. Since these communication links carry important information, they must be protected against unauthorized access, impersonation and other cyber threats.

One important part of protecting satellite communication is cryptographic key management. Cryptographic keys are used by security protocols to encrypt data, verify authenticity or support other security operations. These keys must be generated, distributed, stored, rotated and revoked in a controlled manner. If a key is exposed or remains valid after a satellite is compromised, an attacker may gain unauthorized access to protected communication.

Managing these operations manually becomes difficult when a constellation contains many satellites. Communication opportunities between a satellite and a ground station may also be limited to short contact windows. If a key update is not completed and verified before a contact window closes, the system must handle the incomplete operation safely rather than incorrectly reporting success.

The LEO Satellite Key Management Fabric is proposed as a software system to address these challenges. It will simulate satellite nodes and ground control stations and provide centralized coordination of cryptographic key management. The system is intended to support secure key generation, controlled key distribution, automatic key rotation, satellite identity verification, compromised-node isolation and security event recording.

For example, if Satellite B is identified as compromised during a simulation, the system should revoke its authorization, prevent it from receiving further key updates and record the incident. Similarly, if a communication window closes before a key update is acknowledged, the update should remain pending until the system can safely resume or recover the operation.

In simple terms, the project aims to demonstrate how software can coordinate satellite security operations more consistently, reduce manual key-management work and provide a record of important security events. The proposed prototype will demonstrate these functions in a simulated environment; it will not directly control real satellites or establish operational military-grade security.

1.2 Objectives of the Proposed System

The main objectives of the LEO Satellite Key Management Fabric are:

1. Secure Cryptographic Key Lifecycle Management: Provide controlled procedures for generating, distributing, tracking, rotating and revoking cryptographic keys used by simulated satellite nodes.
2. Satellite Identity Verification: Verify the identity and authorization of nodes before permitting sensitive key-management operations.
3. Efficient Key Updates: Simulate key distribution during short communication contact windows and handle timeouts, interruptions and acknowledgements correctly.
4. Automatic Key Rotation: Support key rotation according to configured time-based or security policies, reducing dependence on manual intervention.

5. Compromised-Node Isolation: Allow authorized security procedures to revoke a compromised node's access and prevent further key distribution to it.
6. Security Auditing: Maintain tamper-evident audit records of key-management requests, authentication outcomes, rotation events, revocations and security incidents.
7. System Monitoring: Provide a dashboard for viewing simulated satellite status, key-management activity, alerts and relevant performance information.
8. Reliability and Maintainability: Support testing, recovery procedures, controlled software updates and ongoing maintenance so that the prototype can be improved without unnecessarily disrupting existing functions.

1.3 Scope of the Problem

The scope of this project covers the design and development of a software-only academic prototype. Its main purpose is to demonstrate the key-management process and associated security controls in a controlled simulation.
The proposed system includes:

- A simulated fleet of satellite nodes and a ground control interface.
- Cryptographic key generation and key lifecycle management.
- Identity verification and authorization checks.
- Secure key-update workflows and automatic rotation policies.
- Simulation of satellite availability and short communication windows.
- Revocation and isolation of compromised simulated nodes.
- Tamper-evident audit logging and security event monitoring.
- A web dashboard for observing system status and operations.
- Automated testing, performance measurement and documented maintenance procedures.

The planned technology stack includes Python, FastAPI, the Python cryptography library, SQLite initially, HTML, CSS, JavaScript, Git, Pytest and Docker. These are planned technologies, not evidence that every component has already been implemented.
The initial scope does not include communication with real orbital satellites, integration with operational military infrastructure or certification for defence use. The prototype also cannot guarantee complete protection against every cyberattack. Its effectiveness must be evaluated through appropriate functional, security and failure-recovery tests.
Future work could investigate larger-scale simulations, stronger protected key storage, integration with approved satellite communication interfaces and independent security assessment. Such extensions would require additional design, infrastructure and validation beyond the initial academic prototype.

2. Identify Key Stakeholders and Challenges

2.1 Stakeholders, Roles and Responsibilities

Although the project is a software prototype, it represents a system involving several types of users and technical responsibilities.
Stakeholder 1: Ground Control Administrators
Register simulated satellite nodes, manage approved policies and initiate authorized key-management operations.
Stakeholder 2: Satellite and Network Operators
Monitor node availability, communication windows and update status; investigate failed or delayed operations.
Stakeholder 3: Cybersecurity Personnel

Define security policies, review authentication failures, assess suspicious activity and coordinate compromised-node isolation.
Stakeholder 4: Cryptographic and Security Engineers
Design and review key lifecycle procedures, cryptographic protocol choices, access controls and recovery mechanisms.
Stakeholder 5: Software Developers
Develop the backend, APIs, simulator, dashboard and supporting services; fix defects and maintain documentation.
Stakeholder 6: Software Testers and Quality Assurance Personnel
Prepare test cases and verify key generation, authentication, rotation, revocation, recovery and system reliability.
Stakeholder 7: Database and System Administrators
Manage stored metadata, audit records, backups, configuration and service availability.
Stakeholder 8: Security Auditors
Review recorded events, examine access-control decisions and verify whether audit records show evidence of unauthorized modification.
Stakeholder 9: Project Manager or Faculty Supervisor
Review project scope, coordinate development activities, evaluate milestones and ensure that the deliverables meet academic requirements.

These stakeholders have different priorities. For instance, an operator needs to know whether a key update succeeded, while a security engineer must verify that the update was authorized and that the key was handled correctly. The software must support both operational visibility and security assurance.

2.2 Major Challenges Faced by Stakeholders
1. Limited Communication Windows
A satellite may not remain connected to a particular ground station long enough to complete every operation. The system must track update progress and distinguish a completed update from one that was interrupted. Otherwise, operators could be shown an incorrect key status.
2. Secure Key Handling
Cryptographic keys are sensitive assets. Unauthorized disclosure, incorrect storage or improper revocation can undermine the security of the communication protected by those keys. The system must use established cryptographic libraries, restrict access to key operations and avoid storing raw keys as ordinary database text.
3. Identity Verification and Unauthorized Access
The system must establish that a node or user is authorized before allowing sensitive operations. Merely registering a satellite in the database must not automatically grant it unrestricted access to key-management services.
4. Compromised-Node Containment
If a simulated satellite is marked as compromised, the system must prevent subsequent key distribution to that node. Revoking access must be enforced by the relevant authorization and key-distribution services, not just represented by a warning on the dashboard.
5. Reliability During Failures
Network interruptions, application errors or database failures may occur while an operation is in progress. The system must handle these conditions without incorrectly reporting success, losing important audit information or leaving security states inconsistent.
6. Audit Integrity and Accountability

Security personnel need reliable records of what happened, when it happened, which node was involved and whether the operation succeeded. A hash-chained audit log can help reveal modifications, but it does not by itself guarantee that records are impossible to alter. Appropriate access restrictions and protected integrity mechanisms are also required.

7. Performance and Scalability

The design should support an increasing number of simulated nodes and key-management requests. As the number of nodes increases, the system must continue to handle requests, record events and display useful status information without unacceptable delays.

8. Software Maintenance and Evolving Threats

Security libraries, dependencies and cryptographic recommendations can change. Developers must be able to correct bugs, apply updates and test changes without accidentally breaking key rotation, access revocation or recovery procedures.

The central challenge is to maintain a consistent and verifiable security process even when communication is intermittent or an operation fails.

3. Analyze the Current Approach

3.1 Existing System or Current Process

The provided project problem statement identifies the need for automated key management across LEO satellite networks, but it does not describe a particular installed key-management system or the exact procedures currently used by a specific organization.

Therefore, for this analysis, the current approach is treated as a reasonable baseline for comparison, not as a claim about every satellite operator.

In a conventional key-management process, authorized personnel or existing security systems may provision cryptographic credentials, apply access policies, schedule key rotation and review security events. Existing communication infrastructure and security protocols may handle parts of authentication and protected data exchange. Depending on the environment, some tasks may be automated while others may require operator intervention.

Such an approach can be effective when the number of nodes and operational conditions are manageable. However, coordinating frequent key updates across a large, moving constellation introduces additional challenges. Updates must be matched to node availability, verified after delivery and handled carefully when communication is interrupted. Security teams also need a consistent way to revoke compromised nodes and review the history of important operations.

The proposed LEO Satellite Key Management Fabric is intended to demonstrate how these responsibilities could be coordinated through a common software platform.

3.2 Strengths of the Current Approach

The existing or conventional approach has several potential strengths:

- Established security procedures: Existing systems may already define how credentials are issued, authenticated, rotated and revoked.
- Human oversight: Authorized personnel can investigate unusual events and make decisions that require operational judgement.
- Use of established cryptographic standards: Mature protocols and libraries can provide well-reviewed security mechanisms when correctly configured.

- Controlled authorization: Restricting sensitive operations to authorized personnel and services can reduce accidental or unauthorized changes.
- Compatibility with existing infrastructure: Established communication and security systems may already be integrated into operational workflows.
- Traceability through existing records: Where logging is implemented properly, event records can help investigate failures and security incidents.

These strengths should be preserved. The proposed software should complement established security mechanisms rather than assume that all current operations are manual or that existing systems are inherently insecure.

3.3 Limitations and Gaps

Several limitations may arise when key management is distributed across separate tools, procedures or services.

Fragmented Key-Management Information: If node status, key metadata, rotation history and security events are maintained separately, personnel may need to consult multiple sources to understand the current state of an update.

Dependence on Manual Coordination: Where rotation or revocation requires manual action, the response may be delayed, especially when many nodes require attention or when a security incident occurs.

Interrupted Updates: A key update that fails during a short contact window must be tracked and recovered safely. Without explicit update states and acknowledgement checks, an incomplete operation could be mistaken for a successful one.

Delayed Incident Response: If revocation procedures are not coordinated across relevant services, a compromised node may continue to have access longer than intended.

Limited Operational Visibility: Without a unified dashboard and consistent event records, it may be difficult to review update failures, authentication problems or the sequence of events leading to an incident.

Difficult Maintenance and Testing: When security functions are tightly coupled or insufficiently documented, changing one component may unintentionally affect another. Automated tests and modular design help reduce this risk.

The main gap addressed by the proposed project is the need for a coordinated, testable software workflow that combines key lifecycle management, node authorization, interruption handling, incident containment and auditability.

The prototype will demonstrate these capabilities under simulated conditions. It will not establish that every real-world satellite system has these limitations or that the proposed design is superior to existing operational solutions without comparative testing.

4. Propose a Suitable Solution Using Software Engineering Principles

4.1 Overall Proposed Software Solution

The proposed solution is a modular web application that coordinates cryptographic key management for simulated satellite nodes. A ground control dashboard will allow authorized users to monitor the simulated fleet and initiate permitted operations. A backend service will process requests, enforce security policies, manage key lifecycle operations and record important events.

The application will be divided into modules so that each component has a clear responsibility. This is important because cryptographic operations, communication simulation, access control and user-interface functions have different requirements and should be testable independently.

The dashboard will communicate with the backend through authenticated API requests. The backend will validate each request before allowing the appropriate service to perform an operation. The simulator will

represent satellite availability and communication interruptions, while the audit service will record relevant security events.
The database will store information such as node identifiers, authorization status, key metadata, policies and event records. Raw cryptographic keys should not be stored as ordinary database text. For the prototype, key material must be handled through a separate protected storage mechanism; a production system would require a stronger key-custody design and could require hardware security modules.

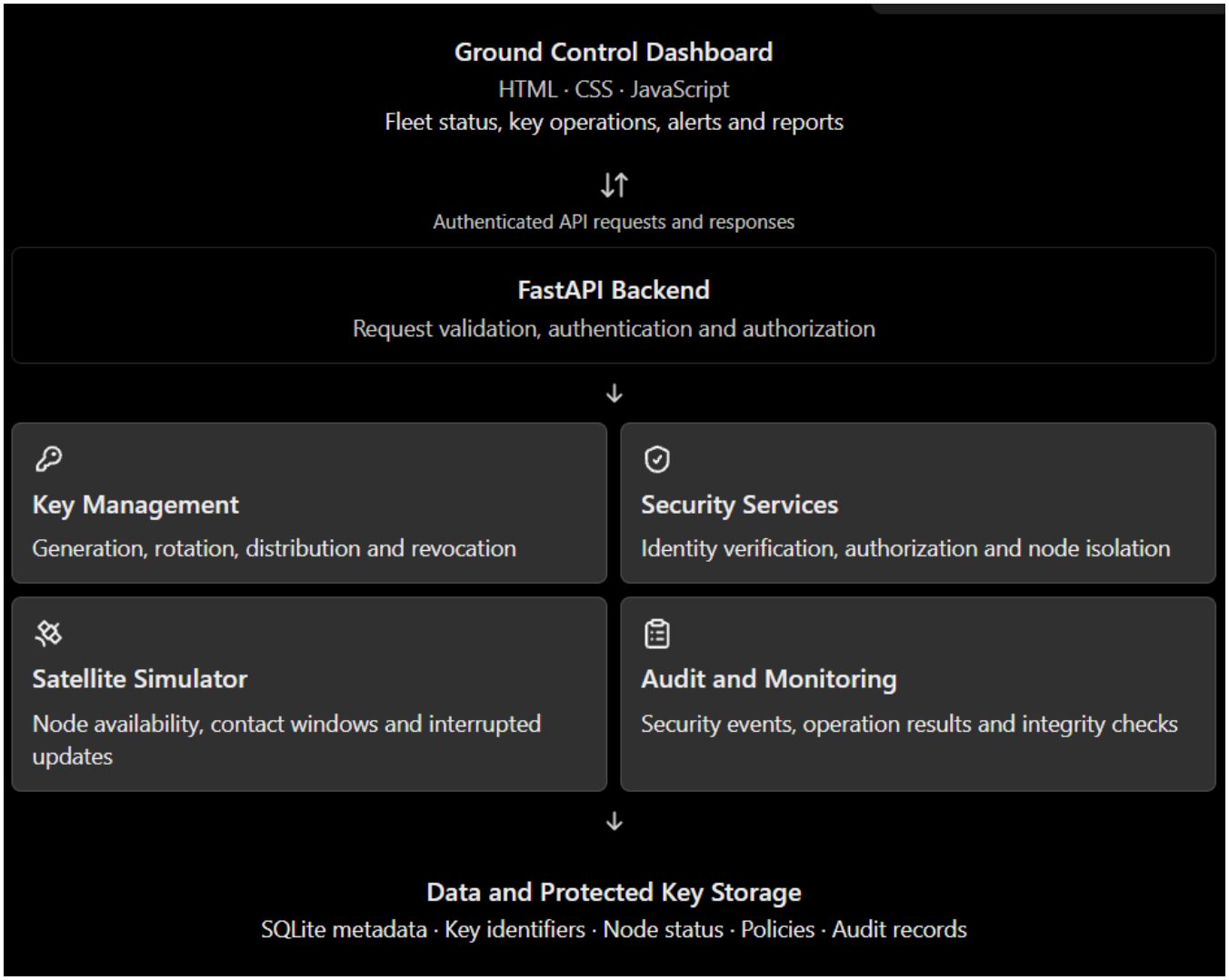


Figure 1. Proposed logical architecture of the LEO Satellite Key Management Fabric. This is a conceptual design, not a claim that the architecture has already been implemented.

4.2 Major Functional Modules

1. Satellite Fleet Management Module
This module will register simulated satellites and maintain information about their identities, availability and authorization status. It will allow authorized users to monitor the fleet through the dashboard.
For example, when a node is marked as unavailable, the dashboard should reflect that status and the update scheduler should not treat the node as ready to receive an update.

2. Cryptographic Key Management Module
This module will manage the lifecycle of cryptographic keys, including generation, assignment, rotation and revocation. It will use established cryptographic libraries rather than a custom-designed encryption algorithm.
The module will track key identifiers, versions and status without unnecessarily exposing the key material.

3. Authentication and Authorization Module
This module will verify the identities of users or simulated nodes and check whether they are permitted to perform requested operations. Authentication establishes identity, while authorization determines what that identity is allowed to do.
For example, a registered node that has been revoked must not be allowed to obtain new key updates simply because its identifier remains in the database.

4. Secure Key Distribution and Contact-Window Simulation Module
This module will model short communication windows and coordinate key-update attempts. It will track whether an update is pending, transmitted, acknowledged, verified or failed.
If a communication window closes during an update, the system should not report success without the required acknowledgement and verification. It should preserve enough state to retry or recover safely when the node becomes available again.

5. Compromised-Node Isolation Module
This module will support an authorized procedure for revoking a compromised node's access and preventing further key distribution to it. It will also record the incident for investigation.
The prototype can demonstrate this by marking a simulated node as compromised and verifying that subsequent key-update requests are rejected.

6. Audit Logging Module
This module will record relevant events, including authentication results, key rotations, revocations, failed updates and node-isolation actions. A hash-chained log can help detect unauthorized changes to previously recorded events.
However, tamper-evident logging is not the same as absolute immutability. Stronger assurance would require protected integrity keys, restricted write access and, where appropriate, independent copies of audit records.

7. Web Dashboard and Alerting Module
The dashboard will present satellite status, update progress, security alerts and audit information in a form that authorized users can understand. It should clearly distinguish successful, pending and failed operations.
This helps users investigate problems without needing to inspect the application's internal code.

8. System Administration and Maintenance Module
This part of the system will support configuration management, user permissions, system-health monitoring, backup procedures and software-version tracking. These functions will help maintain the application as the prototype evolves.

4.3 Functional Requirements

Functional requirements describe what the system must do.

FR1 – Satellite Registration: The system shall register simulated satellite nodes and maintain their identity and status information.

FR2 – Key Management: The system shall generate and manage cryptographic keys using established cryptographic libraries.

FR3 – Authentication and Authorization: The system shall authenticate identities and enforce authorization checks before sensitive operations.

FR4 – Key Rotation: The system shall support key rotation according to configured policies.

FR5 – Contact-Window Simulation: The system shall simulate communication contact windows and track key-update progress.

FR6 – Update Verification: The system shall distinguish successful, pending and failed updates based on the required verification and acknowledgement conditions.

FR7 – Compromised-Node Revocation: The system shall revoke a compromised node's authorization and prevent further key distribution to it.

FR8 – Audit Logging: The system shall record key-management operations and security events in tamper-evident audit records.

FR9 – Dashboard Monitoring: The system shall display node status, update results, alerts and relevant event history through a dashboard.

FR10 – Maintenance Support: The system shall support appropriate monitoring, testing, backup and maintenance procedures.

These requirements provide a basis for later test cases. For instance, FR7 can be tested by revoking a simulated node and verifying that it cannot obtain a subsequent key update.

4.4 Non-Functional Requirements

Non-functional requirements describe the quality and constraints of the system.

1. Security: Protect key material, enforce least privilege, authenticate requests and restrict sensitive operations.
2. Reliability: Handle interruptions and failures without reporting incomplete key updates as successful.
3. Performance: Complete simulated operations within defined timing targets and measure update latency.
4. Scalability: Support an increasing number of simulated nodes without requiring a complete redesign.
5. Maintainability: Keep modules independently testable and document configuration, interfaces and software changes.
6. Usability: Present understandable status information, clear alerts and meaningful operation results.
7. Data Integrity: Maintain consistent key metadata, operation states and auditable event records.
8. Testability: Support repeatable tests for normal operation, unauthorized requests, interrupted updates and recovery.
9. Availability: Detect service failures and provide documented recovery procedures.

Exact performance targets must be established during requirements analysis and measured during testing. We should not claim that the prototype meets a particular latency or availability target until test results support that claim.

4.5 Application of Software Engineering Principles

Modularity: The system separates key management, authentication, simulation, auditing and presentation into distinct components. This makes it easier to identify and correct a problem. For example, a dashboard display defect can be investigated separately from the cryptographic key-generation service.

Scalability: The design keeps responsibilities separate so that data processing, storage and simulation can be improved independently. If the number of simulated satellites increases, the system can be evaluated and optimized without automatically replacing the entire application.

Maintainability: Git will track source-code changes, while automated tests will check existing functionality after modifications. Documentation and controlled releases will help developers understand what changed and why.

Reliability: The update workflow will use explicit states, timeout handling, acknowledgement checks and recovery procedures. If a contact window closes unexpectedly, the system must preserve the distinction between an incomplete operation and a verified successful update.

Usability: The dashboard will show useful information such as node authorization, update status and security alerts. Clear status messages will help users identify whether an operation requires attention.
Security: The system will apply authentication, authorization, protected key handling and restricted access to sensitive operations. A compromised node will be blocked through backend security checks rather than relying solely on its displayed status.

Testability: Each module will have suitable tests. Unit tests will check individual functions, integration tests will check communication between components, and security tests will examine unauthorized access and revocation. Failure simulations will help evaluate recovery behaviour.
Interoperability: Documented APIs will separate the dashboard from backend services. This will make future integration easier, although real satellite or ground-station integration remains outside the initial prototype scope.

Together, these principles support a system that is easier to understand, test, secure and maintain. They are especially important here because a defect in key-update status or authorization could have more serious consequences than an ordinary user-interface error.

5. Justify the Chosen Methodology/Framework

5.1 Recommended Methodology: Agile Scrum with DevOps Practices

Agile Scrum with supporting DevOps practices is a suitable choice for developing the LEO Satellite Key Management Fabric prototype.

The project combines several connected areas: cryptographic key management, identity verification, a satellite simulator, audit logging and a web dashboard. Although the main objectives are known, some implementation details and acceptable performance limits will become clearer only after individual components have been developed and tested.

For example, the initial contact-window simulator may reveal that the update workflow does not correctly handle interrupted transmissions. An iterative development approach allows the team to correct this behaviour, add suitable tests and improve the design before moving on to more advanced features.
Scrum organizes development into short iterations called sprints. At the beginning of each sprint, the team selects prioritized work. At the end, it reviews the completed increment, evaluates test results and identifies improvements for the next sprint.

This approach is appropriate for the academic prototype because it provides regular evidence of progress and allows defects to be addressed before the entire application is complete.

DevOps practices complement Scrum by supporting automated testing, consistent packaging, controlled deployment and monitoring. However, security-sensitive changes should still receive appropriate review and approval. Automated tests passing is useful evidence, but it does not by itself establish that a security-critical change is safe for operational use.

5.2 Suggested Sprint Plan

The following is a proposed development plan, not a record of completed work.

Sprint 1 – Requirements and Planning
- Planned activities: Confirm requirements, scope, stakeholders and acceptance criteria.
- Expected deliverable: Requirements document and initial design.

Sprint 2 – Backend and Satellite Registry
- Planned activities: Establish the backend structure and simulated satellite registry.
- Expected deliverable: Basic API and node-management capability.

Sprint 3 – Key Management and Identity Checks
- Planned activities: Develop and test key lifecycle operations and identity checks.
- Expected deliverable: Initial key-management services.

Sprint 4 – Contact-Window Simulation
- Planned activities: Model contact windows, interrupted updates and recovery states.
- Expected deliverable: Working communication simulation.

Sprint 5 – Security and Audit Logging
- Planned activities: Add authorization enforcement, compromised-node isolation and audit logging.
- Expected deliverable: Core security and accountability functions.

Sprint 6 – Dashboard Development
- Planned activities: Develop the dashboard and connect it to the backend.
- Expected deliverable: Integrated monitoring interface.

Sprint 7 – Testing and Validation
- Planned activities: Conduct unit, integration, security and failure-recovery testing.
- Expected deliverable: Test results and a list of corrected defects.

Sprint 8 – Packaging and Evaluation
- Planned activities: Package the prototype, document deployment and evaluate performance.
- Expected deliverable: Demonstrable prototype and maintenance documentation.

The sequence can be adjusted if testing reveals a major problem. For instance, if the key-update workflow is unreliable, correcting and validating it should take priority over adding more dashboard features.

5.3 Why Scrum Supports Development and Maintenance

Early Delivery of Useful Components: The team can first demonstrate node registration and then add key management, simulation and monitoring incrementally. This makes progress easier to review.

Continuous Feedback: Testing each increment helps reveal unclear requirements and incorrect assumptions. Feedback can be used to improve the next sprint.

Prioritization of Security Risks: Authentication, authorization and safe handling of interrupted updates can be addressed early rather than postponed until the final stage.

Improved Defect Management: Automated tests can be rerun after bug fixes and feature changes. This reduces the risk of introducing regressions into previously working modules.

Adaptability: If a design decision proves unsuitable, the team can revise it based on evidence from tests rather than continuing with an ineffective approach.

Support for Maintenance: Each increment can include documentation, test cases and version-control changes. These records make it easier to diagnose future problems and understand earlier decisions.
DevOps practices strengthen this process by automating repeatable checks and packaging. A typical workflow would run tests and code-quality checks after a proposed change, require review for sensitive changes, and deploy approved releases to a controlled test environment before wider use.
For this project, the combination of Scrum and DevOps provides a practical balance between iterative progress and disciplined software maintenance. It does not remove the need for security review, appropriate testing or controlled release decisions.

6. Discuss Expected Outcomes and Limitations

6.1 Expected Benefits of the Proposed Solution

The proposed LEO Satellite Key Management Fabric is expected to provide the following benefits when its functions are implemented and validated.

1. Reduced Manual Key-Management Effort

Automating key generation, rotation and revocation workflows can reduce the amount of repetitive work required from administrators. It can also make security procedures more consistent by applying configured policies.

2. Better Control Over Key Updates

The contact-window simulator will help demonstrate how updates can be coordinated around limited communication opportunities. Explicit update states and acknowledgement checks can prevent an interrupted operation from being incorrectly recorded as successful.

3. Improved Access Control

Identity verification and authorization checks will help ensure that only permitted users and nodes can perform sensitive operations. Revocation procedures will demonstrate how access can be withdrawn when a node is compromised.

4. Faster Response to Simulated Security Incidents

When a simulated node is marked as compromised, the system can apply the isolation procedure, reject further key-distribution requests and record the event. This will help demonstrate the intended incident-response workflow.

5. Better Auditability

Centralized event records will make it easier to review key rotations, failed updates, authentication results and revocation actions. Tamper-evident logging can also help identify modifications to recorded events.

6. Improved Software Maintainability

A modular architecture, version control and automated testing will make it easier to correct defects and introduce changes without unnecessarily affecting unrelated components.

7. Measurable System Performance

The prototype will allow the team to measure update latency, successful update rates, recovery time and the behaviour of the system during simulated failures. These measurements can be used to identify weaknesses and guide future improvements.

These are expected benefits rather than guaranteed results. Their actual achievement must be demonstrated through implementation, testing and recorded evidence.

6.2 How the Proposed Solution Addresses the Identified Challenges

Challenge 1: Limited Communication Windows
Proposed response: Simulate node availability, track update states and handle interruptions explicitly.
Challenge 2: Unauthorized Access
Proposed response: Apply identity verification and authorization checks before sensitive operations.
Challenge 3: Compromised Satellite Nodes
Proposed response: Revoke authorization and reject further key-distribution requests to isolated nodes.
Challenge 4: Inconsistent Operational Records
Proposed response: Record relevant key-management operations and security events in audit logs.
Challenge 5: Failures During Updates
Proposed response: Use acknowledgements, timeout handling and controlled retry or recovery procedures.
Challenge 6: Difficult Maintenance
Proposed response: Separate components, maintain version history and use automated regression tests.
Challenge 7: Limited Visibility
Proposed response: Provide a dashboard for node status, key-update progress and security alerts.
Challenge 8: Uncertain System Performance
Proposed response: Measure latency, success rates and recovery behaviour during repeatable simulations.
Consider a situation in which Satellite C loses contact with the simulated ground station while a key update is in progress. The proposed system should record the update as incomplete, wait for the conditions required to resume safely and verify the result before reporting success. If the node is subsequently revoked, the authorization service should prevent it from receiving further updates.
This example shows how the proposed design addresses both communication failures and security risks through explicit rules rather than relying only on operator observation.

6.3 Potential Risks, Implementation Challenges and Limitations

1. Limited Realism of the Simulation
A software simulator cannot reproduce every condition experienced by real satellites, including actual orbital communication characteristics, hardware failures and operational network constraints. Successful simulation tests therefore cannot prove that the system is ready for deployment on real satellites.

2. Cryptographic Implementation Errors
Using an established cryptographic library reduces the need to implement low-level cryptographic algorithms independently, but incorrect configuration, key handling or protocol integration can still create vulnerabilities. Security-sensitive operations require careful design and testing.

3. Incomplete Failure Recovery
An interrupted key update may leave the system uncertain about the state of a node. If the workflow does not track acknowledgements and recovery states correctly, the application could display inconsistent information or mishandle a subsequent update.

4. Inadequate Identity or Access Controls
A weakness in authentication or authorization could allow an unauthorized user or node to request sensitive operations. The system must test both successful access and explicit rejection of unauthorized requests.

5. Audit-Log Limitations
A hash-chained audit log can reveal certain modifications, but it cannot guarantee that every record is authentic or that an attacker cannot alter the log and its associated integrity data. Protected keys, access restrictions and independent verification mechanisms may be needed for stronger assurance.

6. Performance Limitations
The prototype may not handle the same number of nodes or requests as a large operational constellation. Performance must be measured under defined test conditions before making claims about scalability.

7. Software Dependency and Maintenance Risks
An updated library or dependency may introduce compatibility problems or change application behaviour. Updates must be reviewed and tested before release.

8. Limited Security Assurance
The prototype will not automatically provide certified military-grade security, immunity to hacking or zero data leakage. A real operational system would require a separate threat assessment, security architecture review, suitable infrastructure, independent validation and applicable certification.

9. Integration Limitations
The initial project will use simulated nodes rather than actual satellite communication systems. Integration with real ground stations, specialized hardware or operational control infrastructure would require additional engineering and authorization.

These limitations do not make the project unsuitable. They define what can reasonably be demonstrated within an academic prototype and what must be investigated before any operational use.

6.4 Risk Mitigation Measures

The following measures can reduce the risks associated with development and evaluation:

- Begin with a simulated environment and test individual modules before integrating them.
- Use established cryptographic libraries and avoid designing a custom encryption algorithm.
- Test authentication, authorization, revocation and interrupted-update scenarios.
- Require acknowledgement and verification before marking an update as successful.
- Keep sensitive key material separate from ordinary application metadata and restrict access to it.
- Protect audit records and test whether unauthorized modifications can be detected.
- Use version control, automated regression tests and reviewed dependency updates.
- Back up relevant data and document recovery procedures.
- Deploy tested versions to a controlled environment before considering any wider release.
- Clearly document the prototype's limitations and avoid claims that exceed the available test evidence.

6.5 Possible Future Enhancements

Once the initial prototype is working and tested, several improvements could be considered.
Larger Constellation Simulation: Increase the number of simulated satellites and test how the system behaves under a higher volume of key-management requests.

Improved Scheduling: Develop more advanced scheduling policies that prioritize key updates according to urgency, node availability and configured security requirements.

Stronger Key Protection: Investigate protected key-storage mechanisms and hardware security modules for environments requiring stronger control over cryptographic key custody.

Enhanced Identity Management: Evaluate stronger certificate-based identities and mutual TLS for authenticated communication between appropriate system components.

Advanced Monitoring: Add more detailed performance metrics, security-event summaries and visualizations of update failures and recovery times.

Improved Audit Assurance: Introduce stronger integrity protection and independently stored audit records were justified by the security requirements.

Automated Testing and Release Checks: Extend continuous integration to run functional tests, security checks and dependency analysis whenever changes are proposed.

Independent Security Evaluation: Arrange a more comprehensive security review and threat assessment before considering integration with real satellite infrastructure.

These enhancements should be prioritized according to their security value, technical feasibility and available resources rather than added simply to increase the number of features.

Conclusion

The LEO Satellite Key Management Fabric is a proposed software-intensive project that addresses the challenges of managing cryptographic keys across a simulated network of Low-Earth Orbit satellites. Its main purpose is to demonstrate how key generation, distribution, rotation, identity verification, compromised-node isolation and audit logging can be coordinated through a common software platform.

The analysis identifies communication interruptions, secure key handling, unauthorized access and reliable incident response as important challenges. The proposed modular architecture addresses these concerns through separate services for key management, authentication, simulation, auditing and monitoring. Functional and non-functional requirements provide a basis for developing and testing these services systematically.

Agile Scrum, supported by DevOps practices, is suitable because the project can be developed in increments, evaluated through repeatable tests and improved as defects or design limitations are discovered. Version control, automated testing and controlled releases will also support long-term maintenance.

The project should be evaluated using evidence from its implementation and testing, including whether unauthorized key requests are rejected, compromised nodes are isolated, interrupted updates are handled correctly and audit modifications can be detected. Performance measurements will help establish the prototype's actual capabilities.

Ultimately, the project's success will depend not simply on the number of features developed, but on whether its security workflows behave correctly under normal and failure conditions. A carefully tested academic prototype can demonstrate the principles of secure satellite key management while providing a foundation for future research and more advanced validation.